MSIAM: High Performance Computing

Lab sheet #3: OpenMP SIMD + Threading

- 2- 2A **submission** is requested for this session.

- **Submission platform:** Only submissions via the Moodle page will be accepted. You can do multiple uploads and the **last upload will be used for grading**. Submissions via Email are not allowed unless there are any technical issues on the submission website.
- **Deadline:** The deadline for the submission is before next lab session on 08.04.2025 1 minute before midnight. This is a **strict** deadline!
- **Programming in groups:** Regarding the programming part, you're encouraged to work in groups of up to 3 students! Please keep in mind that those of you who know the answer should **not tell the answer straight ahead**, but **help others to find the answer "on their own"**.
- **Report/presentation:** A short presentation of up to 15 slides is requested covering the **entire lab sessions** so far (lab 1, 2 and 3). Follow the guidelines given to you throughout the previous labs (make it look professional, no tables of numbers, etc.).
- **Submission format:** For the presentation, provide a **.pdf** file (no Libreoffice/Word or any other format is allowed and will lead to 0 points!).
- **Quality of submission:**
  Regarding the presentation, you can use up to 15 slides. The writing quality, the quality of figures (title, legends, labels), source code, and the readability will be taken into account of the grading. The presentation should be self-explanatory without the source code.

  Regarding the source code, also its **ease of use** and **readability** of the code will be part of the grading. It's better to have less things working correctly rather than having a lot of things "done", but not working correctly.
- **Program specific:** Regarding the code, it must be possible to compile all the code from the main directory in the submitted file just by typing in "`make`".

Université Grenoble Alpes - 2024

Contents

1 Matrix matrix multiplication **2**

1.1 OpenMP Threading 2

1.2 Optional assignments - only this section! 3

1.2.1 MKL 3

1.2.2 Advanced 3

2 Numerical integration **3**

2.1 Test function 4

2.2 Hand-written reduce 4

2.3 OpenMP reduce 4

The company *LOWPerformance* is excited about your work! They want to continue working with you and thought about the following tasks.

Université Grenoble Alpes - 2024

1) Matrix matrix multiplication

This is a continuation of your matrix-matrix multiplication development

1.a) OpenMP Threading

To exploit multi-core architectures, we will use the OpenMP programming model in this assignment.

There are basically two constructs:

`#pragma omp parallel`: This starts a parallel region.

`#pragma omp for`: This can be written in front of a "for" loop in order to execute it in parallel.

`collapse(2)`: This additional attribute can be also used to apply the parallelization to the two following loops instead of only one.

Alltogether, we can also write `#pragma omp parallel for collapse (2)`

> **Assignment** Extend your matrix-matrix multiplication codes with such statements. Benchmark your code on the HPC computers (e.g., using `./run_06_mmul_openmp.sh`) and include these results in your presentation slides.

1.b) Optional assignments - only this section!

1.b.a) MKL

Assignment Implement a function call to `cblas_dgemm` which is, e.g., available in the Intel MKL Library. Compare your results to this library.

1.b.b) Advanced

- (If you have your own computer) Install the Intel compiler (Search for "Intel oneAPI HPC Toolkit") and reinvestigate some previous benchmarks. What can you observe? Hint: You can switch the compiler by setting the environment variable `CXX` to a different compiler, in this case `icpc`.
- Try to use loop unrolling on the main MM-Mul kernel. Unrolling with the Intel compiler can be done with `#pragma unroll(N)` and with the GNU compiler it's `#pragma GCC unroll N`.
- Work on further optimizations and compare your results to the performance of the Intel MKL Library.
- It's possible to (partially) beat the highly optimized Intel MKL! If you're able to do so, let your teacher know about it!

Université Grenoble Alpes - 2024

2) Numerical integration

LOWPerformance is quite impressed by your work and now likes to see how well you can do with a different topic, namely numerical integration.

The value of an integral can be approximated by computing its Riemann sum

$$\int_a^b f(x)dx \approx \sum_{i=1}^{n} f(\tilde{x}_i)\delta x$$

where $\delta x = x_i - x_{i-1} = (b-a)/n$, and $\tilde{x}_i$ is some point in the interval $[x_{-1}, x_i]$, $x_0 = a$ and $x_n = b$

Commonly used sampling points $\tilde{x}_i$ are the left hand, midpoint and right hand rule. LOWPerformance prefers to use the midpoint rule

$$\tilde{x}_i = \left(i + \tfrac{1}{2}\right) \cdot \delta x.$$

2.a) Test function

You are free to choose the function to develop this numerical integration on your own. Keep in mind that we like to use a function of which we actually know the answer for to make sure that our development is right.

An example is given by

$$\int_0^1 \frac{4}{1+x^2} dx$$

for which there is a known analytical solution. Another possibility would be, e.g., to use a very high frequent sin curve with an interval so that its integral is not equal to 0.

Assignment Choose a good function of which you know a solution for to validate and test your development (which will be part of the next assignment).

Université Grenoble Alpes - 2024

2.b) Hand-written reduce

Assignment Develop a program to compute this quadrature up to accuracy 10^{-8}. As a first step, you should develop a version without the OpenMP `reduce` feature.

Comment

You can use the OpenMP `parallel for` pragma.

You will also need some OpenMP runtime library routines such as +10pt+4pt

- `int omp_get_num_threads(); // get the number of threads // currently running in parallel`
- `int omp_get_thread_num(); // get the current thread number`
- `double omp_get_wtime(); // get current wallclock time`
- `int omp_set_num_threads(); // set the total number of threads`

While implementing your program, you should pay attention on how to divide the loop iteration ranges between threads (use the thread ID and the number of threads) and create an accumulator for each thread to hold partial sums. This can be then used outside the parallel region to compute the global sum.

2.c) OpenMP reduce

Assignment +10pt+4pt

- Start with an entirely new program. Feel free to use, e.g., the timing header file of the other program.
- Develop a version with the OpenMP `reduce` construct.
- Benchmark and compare both ways to do the reduce.

Université Grenoble Alpes - 2024